Jiapeng Tang

AI Infrastructure | LLM Inference & GPU Kernel Optimization
+86 158-0628-1370 | 3081129260@qq.com | github.com/tangzzycc

Education

Xi'an Jiaotong University — Master's in Software Engineering Sep 2024 – Jun 2027 (expected)
GPA: **3.82/4.0**; multiple academic scholarships; CET-6.
Research: on-device LLM deployment, mixed-precision inference, and NPU kernel optimization on Qualcomm Snapdragon.
China University of Petroleum (East China) Sep 2020 – Jun 2024
Bachelor's in Software Engineering.

Experience

AMD Physical AI Intern Jul 2026 – Present

vLLM ROCm Model Integration and Optimization

- Integrated CogAgent-18B multimodal inference into vLLM V1 on AMD RDNA 3.5 GPUs. Adapted dual vision encoders and per-layer visual cross-attention, enabled `torch.compile` and HIP Graph, and aligned BF16 outputs with Hugging Face.
- Enabled AWQ W4A16 inference and profiled the execution path with ROCm Profiler. Compared with the initial Hugging Face BF16 eager baseline, reduced TTFT by **47.4%**, achieved **5.28× Decode throughput**, and reduced end-to-end latency by **76.1%**.
- Implemented and tuned Triton LoRA Shrink/Expand kernels for small-batch multi-LoRA inference, increasing Decode throughput by **18.6–24.8%** and reducing end-to-end latency by up to **19.0%**.
- Optimized ROCm/RDNA GEMM and W4A16 kernel dispatch and fixed sliding-window Attention boundaries. Merged vLLM PRs: #53283, #52619, #59550.

FastFlowLM LoRA Inference Support

- Extended FastFlowLM's NPU inference framework with LoRA support for Gemma4-E models. Implemented PEFT adapter weight conversion, hardware-friendly layout transformations, and metadata generation to prepare adapters for NPU execution.
- Implemented runtime adapter loading and adapted Prefill/Decode AIE kernels for LoRA computation. Decoupled adapter loading and computation from base Q4NX weights and supported multiple common LoRA ranks.

On-device Inference Framework Research

- Studied the architecture, inference workflows, and deployment pipelines of GenieX (Qualcomm QAIRT/QNN) and NVIDIA TensorRT-Edge-LLM for on-device LLM inference.

Projects

vLLM Adaptive KV Offload Independent Developer Jun 2026 – Jul 2026

- Addressed high TTFT for short and medium requests that hit external KV cache but still wait in storage queues. Implemented a Load-vs-Recompute policy in vLLM V1 that compares estimated KV load and re-Prefill completion costs and dynamically selects cache reuse or recomputation per request.
- Used GPU profiling and I/O traces to analyze KV promotion, H2D transfers, and GPU Prefill timelines, identifying storage queuing and compute contention. Improved concurrent load/recompute scheduling and added recompute admission control and queued-I/O cancellation to reduce redundant transfers and extra Prefill work.
- Benchmarked Qwen3.6-35B-A3B-FP8 on 4×RTX 5090 (TP4) under a storage-bound mixed workload. For policy-selected 4K latency-sensitive requests, reduced TTFT P50/P99 by **54.7%/36.4%**; overall throughput slightly exceeded the baseline.

Qualcomm Snapdragon LLM Optimization Research Group Project Jan 2026 – Apr 2026

- Deployed Qwen3-1.7B GPTQ W4A16 on Snapdragon 8 Gen 3 and the SA8797 automotive platform through ExecuTorch/QNN. Adapted quantization and graph compilation to import offline GPTQ weights and replace Linear layers, bypassing default online PTQ calibration to avoid requantizing the weights.
- Implemented a custom HVX lookup-table Linear operator, integrating GPTQ weight unpacking, tiled layout transformations, and activation lookup-table computation into the QNN backend. Combined projection fusion, MHA2SHA graph optimization, and 8-bit KV cache to reduce scheduling and memory overhead, improving Decode throughput by approximately **15%** on Snapdragon 8 Gen 3.
- Extended the ExecuTorch/QNN runtime for Qwen3.5 hybrid Attention. Redesigned model state interfaces and Prefill/Decode state transfer to unify KV cache and linear-Attention state management; adapted model export, QNN graph compilation, and the device runner to enable end-to-end Qwen3.5-0.8B FP16 inference.

Technical Skills

- **Programming & Engineering:** C/C++, Python; Linux development and debugging; CMake, Docker, Git; AI coding tools including Codex.
- **Inference & Deployment:** vLLM, ExecuTorch/QNN; Continuous Batching, PagedAttention, quantization, speculative decoding; on-device deployment on Qualcomm and AMD platforms.
- **Kernels & Profiling:** CUDA, HIP, Triton; GPU architecture and memory, kernel fusion and custom operators; performance analysis with Nsight Compute/Systems and ROCm Profiler.